

Time Series Prediction and Anomaly Detection in EEG Data

Christoffer Jensen, 202007962

1. Introduction

Electroencephalogram (EEG) is a very popular method of monitoring brain activity. It is widely used in many areas of clinical work and research. However, one of the issues with using EEG is that the method is very prone to suffer from unwanted noise (Repovs, 2010). It would therefore be desirable to have a method that could detect which parts of an EEG scan contain a large enough amount of noise to compromise the data (anomalies). The aim of this project is to create a deep learning model that would be able to detect whether a part of an EEG scan contains anomalies or not. Since data received from an EEG scan consists of time series, this task lies within the field of sequence models. The solution in this project will be based on an encoder-decoder (Enc-Dec) model which is trained to take in a section of an EEG scan with the middle cut out and then predict this cutout section. Assuming the model learns to predict this cutout, it could in turn be used to detect anomalies in the scans. This could be done by comparing the model's prediction of the cutout to its observed value. If the difference between the prediction and observation is significantly large, it would be marked as containing an anomaly. The main goal of this project is therefore to train such an Enc-Dec model, which given a section of an EEG scan with the middle part cut out can predict the cutout and based on the observed value of the cutout determine if it contains an anomaly.

2. Data and Data Preprocessing

The dataset used in this project comes from the website openneuro.org and is formatted according to the Brain Imaging Data Structure (BIDS). The data was originally collected to be part of a research project on ear-EEG sleep monitoring (Mikkelsen, Villadsen, Otto, & Kidmose, 2017). The data consists of partial polysomnography (PSG) measurements combined with 14 ear-EEG electrodes (channels) from 9 healthy subjects. This project will only use data from 7 of the 14 channels and none of the data from the PSG, to minimize the total amount of data and thereby reduce the computational resources needed. The channels we have chosen are "ELA", "ELE", "ELI", "ERA", "ERG", "ERE" and "ERI". For each subject, two tasks were recorded; one where the subject is awake and one where they are sleeping. The project will only use data from the sleeping task. In this task, each subject was asked to sleep a full night while wearing the monitoring equipment. The EEG time series data was measured in 200Hz over the entire night resulting in a large number of observations per channel. The sample rate of 200Hz results in the individual points not containing much new information compared to neighboring points. As a result, we decided to resample the data to 20Hz. We further found that subject "006" had some issues with

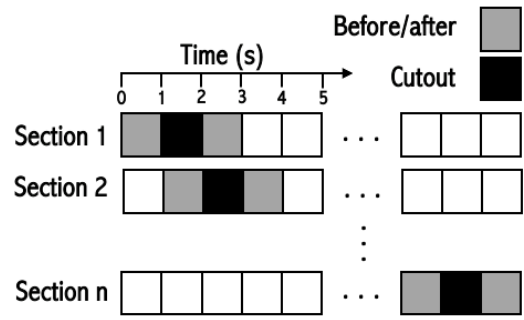


Figure 1: Illustration of method used to divide data into sections.

the equipment leading to their data being compromised. This subject was therefore excluded from the project.

As mentioned in the introduction, the model is going to take in sections of an EEG scan. We define each section to be 3 seconds (or 60 samples after resampling) long. The middle second of each section is going to be the cutout. The sections are made to be overlapping, so the last second of a given section will be the cutout part of the next section as shown in figure 1. The division of the scans into such sections is carried out in our dataset defined as CutoutDataset. Given an EEG time series, this dataset divides the data into sections and returns the sections individually. For each section, it returns the part before the cutout, the part after the cutout flipped, the cutout and the cutout flipped. It flips two of the elements to enable our chosen network structure. This will be described further in section 3. During training of the model, we used k -fold cross-validation. In each fold, we divided the data into a training and validation set. Here, we did one fold per subject, where the validation set would consist of all the data from that one subject and the training set would be all the data from the remaining subjects, resulting in $k=8$. For each fold, we normalized the training data globally across all the subjects to have mean zero and standard deviation 1. The same normalization fitted to the training data would also be used on the validation data. We also tried to normalize the validation data to have mean zero and standard deviation 1, but we found that the model performed better when the normalization fitted to the training data was used. Since the EEG data used in this project doesn't include an anomaly label, we had to include our own anomalies into the original data to test the accuracy of the model's anomaly detection. This will be discussed further in section 5.

3. Network Structure

To solve the described goal, we chose to build a sequence model using the encoder-decoder architecture inspired by the article (Brenner, 2019). The model consists of two encoder-decoder pairs: one moving forward over the part of the input section before the cutout and the other moving backwards over the part after the cutout. Both decoders try to predict the cutout sequence. The outputs of the two encoders are combined through a linear layer to produce one prediction. This structure is also shown in figure 2. We decided to use an encoder-decoder pair in each direction, so the model can use information from both the section before

and after the cutout to make its prediction. Each encoder consists of a three-layer one-directional Long Short Term Memory unit (LSTM) with a hidden size of 1024 and a dropout rate of 0.2. The decoders consist of a one layer one-directional LSTM, also with a hidden size of 1024. During training, we used teacher forcing in the decoder with an initial probability of 0.2. This probability decreases to zero over the first three epochs of training. This setup is shown in figure 3. The specific values of the parameters in the encoder and decoder are decided based on a hyperparameter search which will be discussed further in section 4.

To make the encoder-decoder pairs work in opposite directions, we can use the structure of the output from the CutoutDataset, which is described in section 2. Here we simply define two encoder-decoder pairs, then give the forward pair the first part of the section and the non-flipped targets, and the backward pair the flipped last section and flipped targets. When both pairs have given their prediction, we then flip the backward prediction, so its direction matches the forward prediction. We then concatenate the predictions and use the linear layer to create one prediction.

We have tried many different modifications of this structure during the project but found that the described structure was the best performing. One of the first things we tried was to use Gated Recurrent Unit (GRU) instead of LSTM in the encoder and decoder. Here, we found that GRU struggled to predict the middle parts of the cutout and therefore decided to use LSTM instead since it carries both a hidden and cell state, where the cell state can carry information unchanged quite far over the sequence. After deciding on the LSTM, we tried to add a second linear layer with a ReLU activation between the two linear layers. We also tried to completely replace the linear layer with a weighted sum between the outputs of the two decoders based on the position of the given target. Here, we included the weight as a trainable parameter in the model. None of these two alternatives proved to perform better than the simple linear layer. To avoid any sort of bottleneck when transferring information from the encoder to the decoder and to allow the decoder to focus on the most important parts of the input, we also tried to include attention in the decoder. However, this didn't improve the performance of the model but greatly increased its complexity, so we decided to remove it from the model.

4. Hyperparameter Tuning and Training

As already mentioned, the parameters of the final network structure were decided based on a hyperparameter search. The hyperparameter search is used to define the best values for the non-trainable parameters of the model. Before conducting the hyperparameter search, we must decide on which optimizer and loss function we will utilize. Since we are dealing with a regression problem, we decided to use "mean square error" (MSE) as the loss function. As our optimizer we decided to use the Adam optimizer since this generally has a high performance across different tasks.

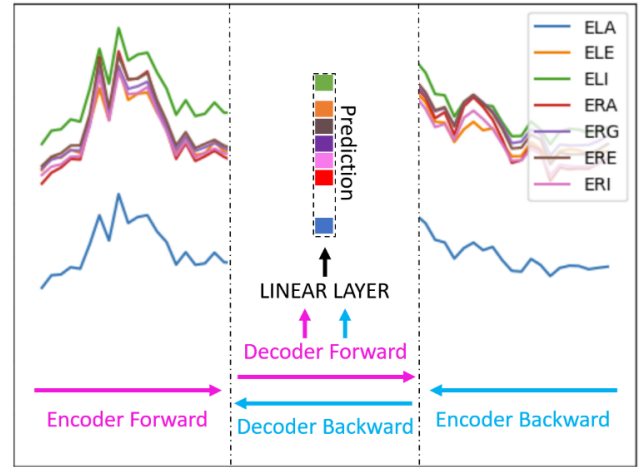


Figure 2: Shows how the model uses the input to make a prediction.

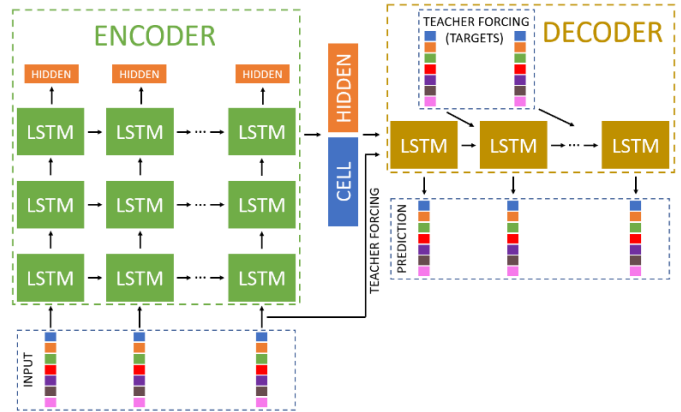


Figure 3: Shows the network structure of the encoder-decoder pairs.

The parameters we choose to include in the hyperparameter search was, hidden size, number of LSTM layers in the encoders, batch size, learning rate, dropout rate in encoder, and initial teacher forcing probability. As mentioned before, the teacher forcing is made to decrease to zero over the first three epochs of training. This is so the decoder gets a good starting point but doesn't learn solely to rely on teacher forcing. We could easily have decided to include more hyperparameters such as resample frequency, epochs with teacher forcing, duration of cutout and sections before / after, etc. This would however significantly increase the computational resources needed to conduct the search. Therefore, several of the hyperparameters were simply assigned a fixed value. In a further attempt to minimize the needed computational resources, we also decided not to use cross-validation when testing the different combinations of hyperparameters. Instead, we defined the same six subjects as training data and the same two subjects as validation data for each combination of parameters. Here, we only used 50% of the data from each of the six training subjects and 20% of the data from each of the two validation subjects. Each combination of hyperparameters was then trained for 10 epochs on the train data and their performance was validated on the validation data. Each combination is only trained for 10 epochs since the combination which will perform best after a full training also tend to be the combination performing best after a few epochs (Smith, 2018). In training, we used a learning rate scheduler set to multiply the learning rate with factor 0.5 after every 3 consecutive epochs with no improvement of validation loss and a minimum learning rate set to 0.0001. All possible combinations of the

values in figure 4 were tested, resulting in a total of 216 combinations. We used neptune.ai to save the result of the different combinations. From the hyperparameter search, we found the optimal values to be those marked with bold in figure 4.

Hyperparameter	Values		
Batch size	512	256	-
Learning rate	0.01	0.001	-
Hidden size	1024	512	128
Encoder dropout prob.	0.2	0.1	0.0
Initial teacher forcing prob.	0.2	0.0	-
LSTM layers in encoder	4	3	2

Figure 4: Shows which hyperparameters and their values used in the search. The optimal values are marked with **bold**.

After conducting the search, we trained the model with the structure as described in section 3 and the hyperparameter values decided by the search. As described in section 2, we used 8-fold cross-validation, one per subject, to train the model and we also used the same learning rate scheduler as in the hyperparameter search. For each fold, we trained the model for 60 epochs. The average MSE loss on the validation data for the last epoch for each fold is shown in figure 5. The result of the cross-validation is also saved using neptune.ai.

5. Performance

To evaluate both the prediction and anomaly detection performance of the Enc-Dec model, we use a linear model as a benchmark. The linear model simply draws a straight line from the last point of the section before the cutout to the first point in the section after the cutout. This is done for each channel. For each subject, we calculated the average MSE loss using the linear benchmark. The result is shown in figure 5. For both the Enc-Dec model and the benchmark, a weighted average of the average MSE loss for each subject is calculated. For each subject, the loss is weighted based on how large a fraction of the total amount of data the validation subject accounts for. The result is shown in figure 5.

Fold / val. subject	Train loss	Validation loss	
	Enc-Dec	Benchmark	Enc-Dec
1	0.066	0.062	0.210
2	0.044	0.007	0.006
3	0.042	0.085	0.110
4	0.054	0.026	0.032
5	0.047	0.309	0.481
7	0.037	0.298	0.297
8	0.039	0.057	0.094
9	0.044	0.126	0.069
Weighted avg.	0.047	0.122	0.170

Figure 5: Shows the result of validating the Enc-Dec and benchmark model for each subject and the weighted sum of each model.

The results in figure 5 show that the simple linear model is better at predicting the cutout part of the section than the Enc-Dec model. From the figure, we further see that the validation loss for the Enc-Dec model varies significantly between the different subjects. This shows that the model doesn't seem to generalize well across all subjects. If we

look at figure 6, which shows the Enc-Dec model's prediction of a section when validated on subject "009", we further see that the model doesn't seem to capture a significantly larger amount of information than a simple linear prediction. This is the general trend when inspecting the predictions made by the model.

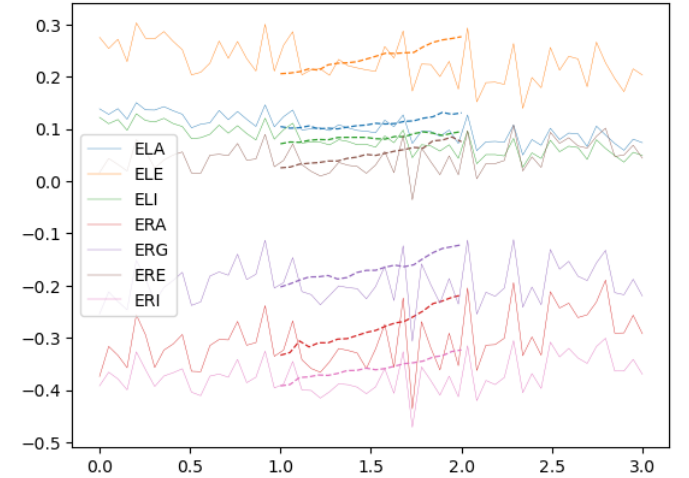


Figure 6: Shows prediction made by the Enc-Dec model for section belonging to subject "009".

The next step is to evaluate the model's performance at anomaly detection. The original data doesn't have an anomaly label and we therefore created a new dataset called AnomalyDataset. This functions as a "wrapper" for the CutoutDataset, meaning it returns sections as CutoutDataset but now adds some anomaly in form of a random function to the sections with probability 0.25. It further adds an anomaly target for each section. This target is a boolean stating "True" if the section has an anomaly added and "False" if not. As mentioned in the introduction, a section is defined as containing an anomaly if the difference between the model's prediction and the observed value is significantly large. We therefore need to define what constitutes a "significantly large" difference. For the Enc-Dec model, this is done by training the model on data from the subjects "001", "002", "004", "005", "007" and "009" without anomalies added and then use this trained model to predict the cutout of all the sections for subject "003" again without anomalies. For each prediction, the MSE loss is saved. Then, the trained model is now used to predict anomalies for subject "008" where a section is defined as an anomaly if its MSE loss is greater than the 0.95-quantile of all the MSE losses from subject "003". The same process is carried out for the benchmark model without the training step. The results of the anomaly detections can be seen in figure 7.

<u>Encoder-Decoder</u>				<u>Benchmark</u>			
PREDICTION				PREDICTION			
TARGET		False	True	TARGET		False	True
	False	17583	332		False	17580	335
	True	5341	753		True	4959	1135

Figure 7: Confusion matrix of the anomaly detection results.

In figure 7, we see that the linear benchmark also performs better at anomaly detection compared to the Enc-Dec model. We further see that the Enc-Dec model has a higher

tendency to classify the sections as not containing an anomaly.

6. Discussion

As presented in section 5, our Enc-Dec model doesn't outperform a simple linear model in neither prediction nor anomaly detections of EEG scans. There are however parts of our process where there is room for possible improvements. One such part is the hyperparameter search. Many of the hyperparameters in the project have just been chosen to have some fixed value without any greater reasoning for this value than common sense. This is parameters such as length of cutout and section before / after, epochs with teacher forcing, resample frequency etc. If these were to be included in the search, it must be expected to improve the overall performance of the model. In the hyperparameter search, we further only use the same two subjects as validation for all combinations to save computational resources. Consequently, we risk overfitting the model to these subjects. It would be expected to yield better results if we instead used cross-validation in the search. Another place for improvement is the data preprocessing. As mentioned, some data preprocessing has been included in the project such as resampling the frequency, exclusion of subject "006", normalization etc. There is however room for a more extensive preprocessing. When examining the MSE loss of the model for each section, we see that some of the sections have an MSE of over 50, even after normalization. This is caused by these specific sections having an extreme variation in some of the channels. An example of such a section is show in figure 8. In the original study which the data was collected for, they rejected 19% of the channels because they found that they included a significant amount of noise. If the same thorough preprocessing had been applied in this project, it could potentially have improved the performance. In figure 5, we saw that the model doesn't generalize well across the different subjects. This could be caused by the fact that we only have data from 8 different subjects. The small number of subjects might lead to personal trends in the data affecting the model's performance when generalized to new subjects. If computational resources weren't limited, then this generalization problem could possibly be fixed by including a greater amount of data across a larger number of subjects when training the model. It could however also be argued that given the poor performance of the current model it would be more beneficial to rethink the network structure before utilizing more data in the training process.

As mentioned in section 2, the data used for training the Enc-Dec model doesn't include an anomaly label. This further introduces a problem since the data very likely does contain anomalies of the sort we are trying to predict. If the model is trained on data containing anomalies, then its prediction of unknown data is also likely to be affected by these anomalies. This could consequently result in the predictions from the Enc-Dec model taking the form of data with

anomalies. This would in turn compromise the model's anomaly detection accuracy since its predictions then would be closer to the observed value if the observed value contains an anomaly. To solve this problem, the model would need to be trained on data which is known to be anomaly free.

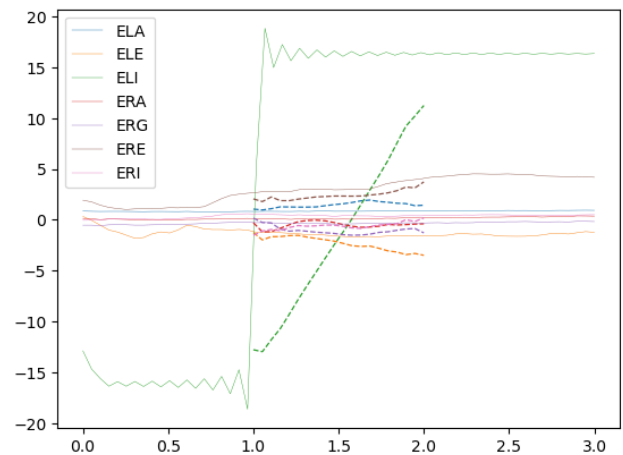


Figure 8: Prediction of section with high variation in one of its channels.

7. Conclusion

The main goal of this project was to train a sequence model using the encoder-decoder architecture to predict EEG scans and thereby enabling anomaly detection of the scans. We defined a benchmark model to evaluate the success of our Enc-Dec model. Even though the benchmark was a simple linear model, the Enc-Dec model couldn't outperform it in neither prediction nor anomaly detection. There is however room for improvement in the process used in the project. Specifically, the hyperparameter search and data preprocessing could be improved. In general, the project could probably benefit from having more data from a larger number of different subjects. Furthermore, if any anomalies in the training data were labeled it could possibly help to improve the accuracy of anomaly detection.

Bibliography

- Brenner, M. (2019). *Implementing Seq2Seq Models for Efficient Time Series Forecasting*. Retrieved from <https://medium.com/@maxbrenner-ai/implementing-seq2seq-models-for-efficient-time-series-forecasting-88dba1d66187>
- Mikkelsen, K. B., Villadsen, D. B., Otto, M., & Kidmose, P. (2017). *Automatic sleep staging using ear-EEG*. Retrieved from <https://biomedical-engineering-online.biomedcentral.com/articles/10.1186/s12938-017-0400-5>
- Repovs, G. (2010). *Dealing with Noise in EEG Recording and Data Analysis*. Retrieved from chrome-extension://efaidnbmnnnibpcajpglclefindmkaj/https://ims.mf.uni-lj.si/ims_archive/15-01/15b-21.pdf
- Smith, L. N. (2018). *A disciplined approach to neural network hyper-parameters: Part 1*. Retrieved from <https://arxiv.org/abs/1803.09820>